Course: COMP5313 - Artificial Intelligence
Exercise 1: Use Multiple Perceptrons to Design XNOR Logic Circuit
Student: Sumit Rajendra Bhosle - 1366816

1. Program Objective

The objective of this exercise is to implement an **XNOR logic circuit using multiple perceptrons** in Python/Jupyter Notebook.

XNOR produces 1 when both inputs are the same and 0 when they are different:

Input 1	Input 2	XNOR
0	0	1
0	1	0
1	0	0
1	1	1

Since XNOR is not linearly separable, multiple perceptrons are used instead of a single perceptron. This follows the exercise requirement to use a multi-perceptron architecture.

2. Program Approach

The program was implemented in two stages:

Stage 1 - Manual Perceptron Implementation

- Implemented a step-function perceptron.
- Created and tested basic logic gates using manually selected weights and biases.
- Connected multiple perceptrons to construct the XNOR circuit.
- Tested different weights and biases manually.

Stage 2 - Automatic Training

- A sigmoid activation function was introduced for gradient-based training.
- Forward propagation was implemented through the hidden and output layers.
- Backpropagation was used to calculate errors and gradients.
- Gradient descent was then used to automatically update the weights and biases.
- The network was trained for multiple epochs and then tested against the XNOR truth table.

The lecture material also describes backpropagation as passing the error gradient backward through the layers using the chain rule.

3. XNOR Circuit Development

During the manual implementation, the initial combination of:

NAND → OR → AND

produced the **XOR** output instead of XNOR.

The result was therefore analysed and corrected. The final output perceptron was changed from **AND to NAND**, giving the working architecture:

NAND → OR → NAND

This produced the required XNOR output:

[1 0 0 1]

This short development step was kept in the notebook to show how the circuit was tested and corrected.

4. Training Result

After automatic gradient-descent training, the network produced:

Target XNOR Output:

[1 0 0 1]

Final Network Output:

[0.97418027 0.02961324 0.02394144 0.97704906]

Predicted XNOR Output:

[1 0 0 1]

The final mean squared error was approximately:

0.00066

The predicted binary output therefore matches the required XNOR truth table for all four input combinations.

5. ScreenShots :

OR GATE

[20]:
```
# OR gate

or_weights = np.array([1,1])
or_bias = -0.5

for x in X:
    output = perceptron(x,or_weights,or_bias)
    print(x,"->",output)
```

```
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 1
```

AND GATE

[21]:
```
# AND gate

and_weights = np.array([1,1])
and_bias = -1.5

for x in X:
    output = perceptron(x,and_weights,and_bias)
    print(x,"->",output)
```

```
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1
```

NAND GATE

[22]:
```
# NAND gate

nand_weights = np.array([-1,-1])
nand_bias = 1.5

for x in X:
    output = perceptron(x,nand_weights,nand_bias)
    print(x,"->",output)
```

```
[0 0] -> 1
[0 1] -> 1
[1 0] -> 1
[1 1] -> 0
```

HIDDEN PERCEPTRONS

[24]:
```
# HIDDEN PERCEPTRON 1 : NAND

nand_output = []

for x in X:
    output = perceptron(x,nand_weights,nand_bias)
    nand_output.append(output)
nand_output = np.array(nand_output)

print("NAND OUTPUTS: ",nand_output)
```

```
NAND OUTPUTS:  [1 1 1 0]
```

[25]:
```
# HIDDEN PERCEPTRON 2 : OR
or_output = []

for x in X:
    output = perceptron(x,or_weights,or_bias)
    or_output.append(output)
or_output = np.array(or_output)

print("OR OUTPUTS: ",or_output)
```

```
OR OUTPUTS:  [0 1 1 1]
```

[54]:
```
# MANUAL TEST 1 WITH DIFFERENT WEIGHTS AND BIAS

test_weights = np.array([2,2])
test_bias = -1

test_output = []

for x in X:
    output=perceptron(x,test_weights,test_bias)
    test_output.append(output)

test_output = np.array(test_output)

print("weights: ", test_weights)
print("bias :", test_bias)
print("Output :", test_output)
```

```
weights:  [2 2]
bias : -1
Output : [0 1 1 1]
```

[85]:
```
# update Weights and Biases using gradient Descent

learning_rate = 0.1

w2 = w2 - learning_rate * dw2
b2 = b2 - learning_rate * db2

w1 = w1 - learning_rate * dw1
b1 = b1 - learning_rate * db1

print("Updated w1:")
print(w1)

print("\nUpdated b1:")
print(b1)

print("\nUpdated w2:")
print(w2)

print("\nUpdated b2:")
print(b2)
```

```
Updated w1:
[[ 0.41182114  0.1254733 ]
 [-0.6338754  -1.46658319]]

Updated b1:
[-0.56100715 -0.26781879]

Updated w2:
[-0.15390299 -0.51586205]

Updated b2:
-0.20034009042862647
```

Automated Test

[87]:
```
# Test the trained XNOR network

z1 = np.dot(X, w1) + b1
hidden_output = sigmoid(z1)

z2 = np.dot(hidden_output, w2) + b2
final_output = sigmoid(z2)

predicted_output = (final_output >= 0.5).astype(int)

print("Target XNOR Output:")
print(y)

print("\nFinal Network Output:")
print(final_output)

print("\nPredicted XNOR Output:")
print(predicted_output)
```

```
Target XNOR Output:
[1 0 0 1]

Final Network Output:
[0.97418027 0.02961324 0.02394144 0.97704906]

Predicted XNOR Output:
[1 0 0 1]
```